

Cryptography, in turn, is a specialised branch of cybersecurity. It involves securing information through mathematical techniques and algorithms, ensuring data confidentiality, integrity, and authenticity. Cryptography works by encoding data to prevent unauthorised access or tampering. In this series, we will focus primarily on cryptographic methods that can be implemented using SageMath.

Now, let us explore the important fields of study in cryptography. These include classical encryption schemes, symmetric key cryptography (also known as private key cryptography or secret key cryptography), asymmetric key cryptography (also known as public key cryptography), digital signatures, key exchange protocols, hash functions, zero-knowledge proofs, post-quantum cryptography, steganography, quantum cryptography, etc. We will cover many of these topics within the scope of this series in a straightforward manner to ensure that everyone can follow along and engage with the discussion.

Before we proceed, it is important to understand two key terms: encryption and decryption. Encryption is the process of transforming readable data, known as plaintext, into an unreadable format called ciphertext. This ensures that the data remains confidential and protected from unauthorised access. It also safeguards the data's integrity and authenticity, ensuring it has not been altered or tampered with. Decryption is the reverse process, where the ciphertext (unreadable data) is converted back into plaintext (readable data). This allows the intended recipient to access the original message securely.

In cryptographic discussions, fictional characters are often used to illustrate various scenarios and algorithms. The classic names include Alice (A): the sender or initiator of secure communication, and Bob (B): the recipient or responder. Eve (E) represents an eavesdropper attempting to intercept or compromise the communication, while Mallory (M) stands for a malicious attacker, often illustrating active attacks. For example, when Alice wants to send a secure message to Bob, Eve may try to intercept it, or Mallory might attempt to alter it. Additional characters include Charlie (C), sometimes used as an intermediary or third party, and Dave (D), who may represent an additional participant. Trent (T) often represents a trusted third party or authority, while Victor (V) is used in scenarios involving zero-knowledge proofs and acts as the verifier. Although I considered using Indian names to add a local flavour, I refrained because all the classic cryptography textbooks use names like Alice and Bob. We will follow this convention throughout our discussions to maintain consistency with the standard references.

Cryptography involves a lot of mathematics, but my aim is to make this discussion as accessible as possible with minimal math. So, before we dive into the mathematical

aspects of cybersecurity, let me start with a story. Alice wants to send a secret message to Bob. She has a box, along with a lock and key. Alice puts the secret message in the box, locks it with her key, and sends the box to Bob through Trent. However, Bob can't open the box because he doesn't have the key. Now, Alice also needs to send the key to Bob through Trent. But here is the problem: Can Alice really trust Trent? What if Trent opens the box and reads the secret message? Sending both the locked box and the key with the same person isn't always safe.

Realising this, Alice tries a different strategy. She locks the box with her key and sends it to Bob. Bob, unable to open it, adds his own lock to the box and sends it back to Alice. Alice unlocks her lock and sends the box back to Bob, who can now unlock it with his own key and open the box. In this way, the keys are never transported, but the box has to be exchanged multiple times between Alice and Bob. This simple story highlights many of the key challenges in cryptography: sending secret messages, managing keys, and trusting third parties. In this series, we will explore how cryptographic algorithms and SageMath methods can solve these challenges efficiently.

Classical encryption schemes

Let us begin our discussion with classical encryption schemes. These are called 'classical' because they have been around for a very long time. For example, the Caesar cipher, named after Julius Caesar (ruler of Rome from 49 BC to 44 BC), has been in use for over two millennia. It is important to note that Julius Caesar didn't invent the Caesar cipher; he simply popularised it by using it in his personal correspondence.

Caesar cipher

The Caesar cipher is one of the simplest ciphers that has ever existed. Most of us are familiar with it and may have even used it for fun during our school days. To demonstrate how it works, let us use the string 'I READ OSFY' as our plaintext. In a Caesar cipher, each letter in the plaintext is shifted by a certain number of positions down the alphabet. In this case, we will shift every letter three positions forward (Julius Caesar himself used a shift of three positions). For example, A becomes D, B becomes E, C becomes F, and so on. Let us go through the encryption step by step. The letter 'I' shifts to 'L' ($I \rightarrow J \rightarrow K \rightarrow L$), 'R' shifts to 'U' ($R \rightarrow S \rightarrow T \rightarrow U$), 'E' shifts to 'H' ($E \rightarrow F \rightarrow G \rightarrow H$), and so on. But what happens with the letter 'Y'? Since it is at the end of the alphabet, when we shift it forward three positions, we wrap around to the beginning of the alphabet. This is like how a clock works: after 12 o'clock, the next hour is 1, not 13. So, 'Y' shifts to 'B' ($Y \rightarrow Z \rightarrow A \rightarrow B$). This wrap-around effect is an example of modular arithmetic, a concept often used in cryptography. It means that after reaching the maximum letter (Z), we start